

Математическое моделирование

Лабораторная работа № 6

Суннатилло Махмудов

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Теоретические сведения	8
4	Выполнение лабораторной работы	9
4.1	Теоретические сведения	9
4.2	Задача	12
5	Численное решение трёх задач Коши и визуализация	14
5.1	Инициализация проекта и загрузка пакетов	14
5.2	Общие начальные условия	15
5.3	Первая модель	15
5.4	Вторая модель	15
5.5	Третья модель	16
5.6	Утилита: один прогон модели	17
5.7	Запуск первой модели	18
5.8	Запуск второй модели	19
5.9	Запуск третьей модели	19
5.10	Вывод первых строк таблиц	20
5.11	Показать график в интерактивной среде	20
6	Параметрическое исследование трёх дифференциальных моделей	21
6.1	Активация проекта и загрузка пакетов	21
6.2	Установка каталогов	21
6.3	Определение моделей	22
6.4	Выбор модели по типу эксперимента	22
6.5	Правая часть уравнения для анализа производной	23
6.6	Базовые параметры первой модели	23
6.7	Базовые параметры второй модели	24
6.8	Базовые параметры третьей модели	25
6.9	Функция для запуска одного эксперимента	26
6.10	Функция печати результатов одного эксперимента	29
6.11	Функция построения графика $n(t)$	30

6.12	Функция построения графика производной dn/dt	31
6.13	Функция построения фазовой траектории dn/dt от n	32
6.14	Запуск базовых экспериментов	33
6.15	Параметрическое сканирование	34
6.16	Запуск всех экспериментов	37
6.17	Анализ результатов сканирования	40
6.18	Сравнительные графики $n(t)$ для каждой модели	40
6.19	Сравнительные графики dn/dt для каждой модели	42
6.20	Сравнительные фазовые траектории $dn/dt(n)$	43
6.21	График зависимости n_final от параметра a	45
6.22	График зависимости n_final от параметра b	46
6.23	График зависимости n_max от параметра a	47
6.24	График зависимости n_max от параметра b	48
6.25	График зависимости $saturation_final$ от параметра a	49
6.26	График зависимости $saturation_final$ от параметра b	50
6.27	Бенчмаркинг	51
6.28	График времени вычисления от параметра a	53
6.29	График времени вычисления от параметра b	54
6.30	Сохранение всех результатов	55
6.31	Базовые эксперименты	56
6.32	Сравнение базовых экспериментов	65
6.33	Параметрическое сканирование	66
6.34	Бенчмаркинг	73
6.35	Выводы	76

Список литературы

78

Список иллюстраций

4.1	График решения уравнения модели Мальтуса	11
4.2	График логистической кривой	12
6.1	Первая модель: зависимость $n(t)$	56
6.2	Первая модель: скорость изменения	57
6.3	Первая модель: фазовая траектория	58
6.4	Вторая модель: зависимость $n(t)$	59
6.5	Вторая модель: скорость изменения	60
6.6	Вторая модель: фазовая траектория	61
6.7	Третья модель: зависимость $n(t)$	62
6.8	Третья модель: скорость изменения	63
6.9	Третья модель: фазовая траектория	64
6.10	Зависимость итогового значения n от параметра a	66
6.11	Зависимость итогового значения n от параметра b	67
6.12	Зависимость максимального значения n от параметра a	68
6.13	Зависимость максимального значения n от параметра b	70
6.14	Зависимость финального насыщения от параметра a	71
6.15	Зависимость финального насыщения от параметра b	72
6.16	Зависимость времени решения ODE от параметра a	73
6.17	Зависимость времени решения ODE от параметра b	75

Список таблиц

1 Цель работы

Изучить модель эффективности рекламы

2 Задание

1. Изучить модель эффективности рекламы
2. Построить графики распространения рекламы в заданных случаях
3. Определить для случая 2 момент времени, в который скорость распространения рекламы будет максимальной

3 Выполнение лабораторной работы

3.1 Теоретические сведения

4 Выполнение лабораторной работы

4.1 Теоретические сведения

Организуется рекламная кампания нового товара или услуги. Необходимо, чтобы прибыль будущих продаж с избытком покрывала издержки на рекламу. Вначале расходы могут превышать прибыль, поскольку лишь малая часть потенциальных покупателей будет информирована о новинке. Затем, при увеличении числа продаж, возрастает и прибыль, и, наконец, наступит момент, когда рынок насытится, и рекламировать товар станет бесполезным.

Предположим, что торговыми учреждениями реализуется некоторая продукция, о которой в момент времени t из числа потенциальных покупателей N знает лишь n покупателей. Для ускорения сбыта продукции запускается реклама по радио, телевидению и других средств массовой информации. После запуска рекламной кампании информация о продукции начнет распространяться среди потенциальных покупателей путем общения друг с другом. Таким образом, после запуска рекламных объявлений скорость изменения числа знающих о продукции людей пропорциональна как числу знающих о товаре покупателей, так и числу покупателей о нем не знающих

Модель рекламной кампании описывается следующими величинами. Считаем, что $\frac{dn}{dt}$ - скорость изменения со временем числа потребителей, узнавших о товаре и готовых его купить, t - время, прошедшее с начала рекламной кампании, N - общее число потенциальных платежеспособных покупателей, $n(t)$ - число уже информированных клиентов. Эта величина пропорциональна

числу покупателей, еще не знающих о нем, это описывается следующим образом $\alpha_1(t)(N - n(t))$, где $\alpha_1 > 0$ - характеризует интенсивность рекламной кампании (зависит от затрат на рекламу в данный момент времени). Помимо этого, узнавшие о товаре потребители также распространяют полученную информацию среди потенциальных покупателей, не знающих о нем (в этом случае работает т.н. сарафанное радио). Этот вклад в рекламу описывается величиной $\alpha_2(t)n(t)(N - n(t))$. эта величина увеличивается с увеличением потребителей узнавших о товаре.

Математическая модель распространения рекламы описывается уравнением:

$$\frac{dn}{dt} = (\alpha_1(t) + \alpha_2(t)n(t))(N - n(t))$$

При $\alpha_1(t) \gg \alpha_2(t)$ получается модель типа модели Мальтуса, решение которой имеет вид

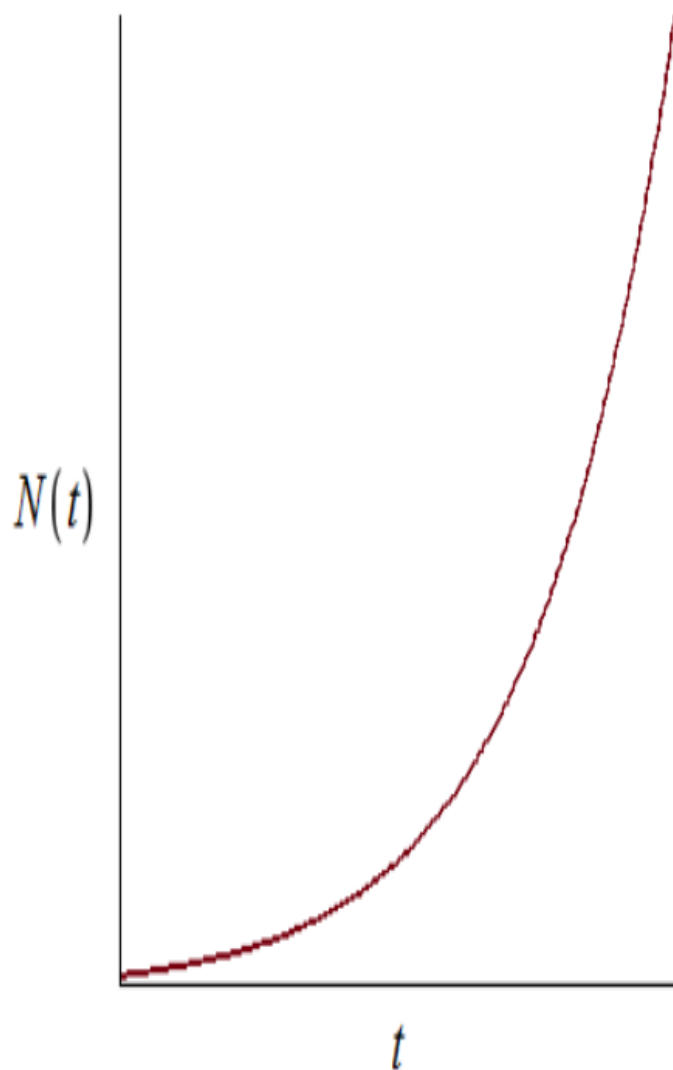


Рисунок 4.1: График решения уравнения модели Мальтуса

В обратном случае $\alpha_1(t) \ll \alpha_2(t)$ получаем уравнение логистической кривой

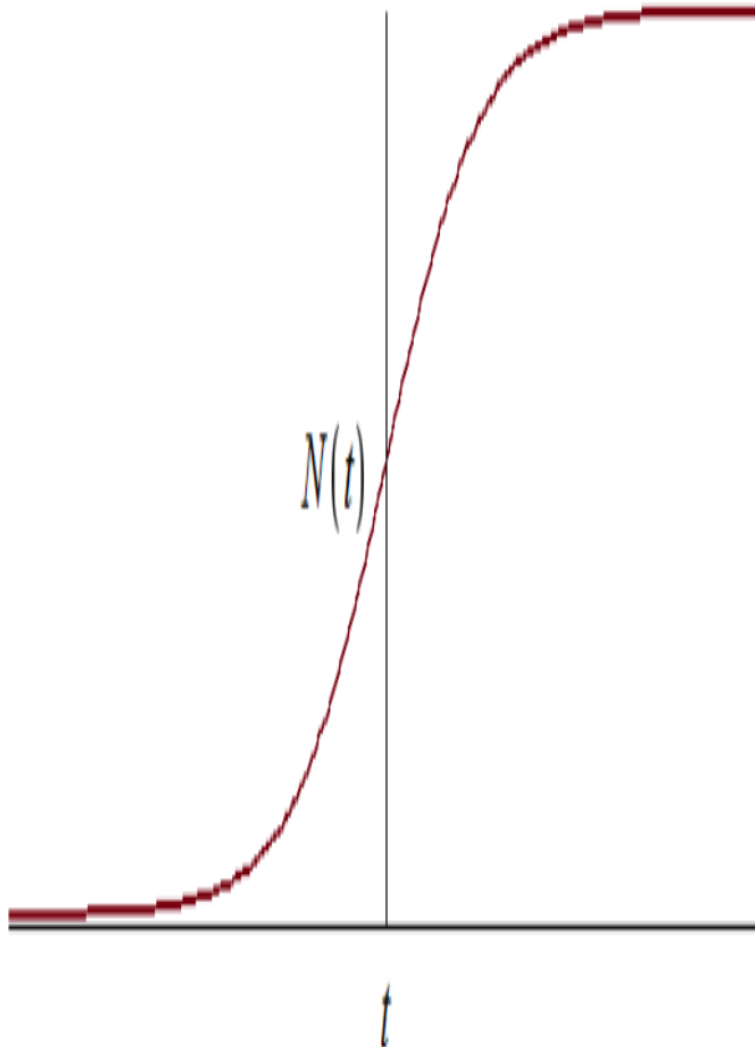


Рисунок 4.2: График логистической кривой

4.2 Задача

Постройте график распространения рекламы, математическая модель которой описывается следующим уравнением:

1. $\frac{dn}{dt} = (0.54 + 0.00016n(t))(N - n(t))$
2. $\frac{dn}{dt} = (0.000021 + 0.38n(t))(N - n(t))$

3. $\frac{dn}{dt} = (0.2 * \cos t + 0.2 \cos 2tn(t))(N - n(t))$

При этом объем аудитории $N = 609$, в начальный момент о товаре знает 4 человек.

Для случая 2 определите в какой момент времени скорость распространения рекламы будет иметь максимальное значение.

Для моделирования процесса и построения графиков использовались внешние файлы с программным кодом:

5 Численное решение трёх задач

Коши и визуализация

Цель: решить три варианта дифференциального уравнения, построить графики $n(t)$, сохранить изображения и таблицы с результатами.

5.1 Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using Plots
using DataFrames
using CSV
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

5.2 Общие начальные условия

```
N = 609.0  
n0 = 4.0  
u0 = [n0]
```

5.3 Первая модель

Уравнение: $dn/dt = (a + bn)(N - n)$

```
a1 = 0.54  
b1 = 0.00016  
  
t0_1 = 0.0  
tmax_1 = 10.0  
tspan_1 = (t0_1, tmax_1)  
  
p1 = (a = a1, b = b1, N = N)  
  
function model_case_1!(dy, y, p, t)  
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])  
end
```

5.4 Вторая модель

Уравнение: $dn/dt = (a + bn)(N - n)$

```

a2 = 0.000021
b2 = 0.38

t0_2 = 0.0
tmax_2 = 0.04
tspan_2 = (t0_2, tmax_2)

p2 = (a = a2, b = b2, N = N)

function model_case_2!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end

```

5.5 Третья модель

Уравнение: $dn/dt = (a \cos(t) + b \cos(2t)n) * (N - n)$

```

a3 = 0.2
b3 = 0.2

t0_3 = 0.0
tmax_3 = 0.15
tspan_3 = (t0_3, tmax_3)

p3 = (a = a3, b = b3, N = N)

function model_case_3!(dy, y, p, t)
    dy[1] = (p.a * cos(t) + p.b * cos(2 * t) * y[1]) * (p.N - y[1])
end

```

5.6 Утилита: один прогон модели

```
function run_case(case_name; model!, u0, tspan, p, nt = 500, image_
```

— Сетка по времени —

```
tgrid = collect(LinRange(tspan[1], tspan[2], nt))
```

— Решение ОДУ —

```
prob = ODEProblem(model!, u0, tspan, p)  
sol = solve(prob, Tsit5(), saveat = tgrid)
```

— Таблица с результатами —

```
df = DataFrame(  
    t = sol.t,  
    n = [u[1] for u in sol.u]  
)
```

— График $n(t)$ —

```
plt = plot(  
    sol,  
    xlabel = "t",  
    ylabel = "n(t)",  
    label = "n(t)",  
    lw = 2,  
    title = "XXXXXXXXXX XXXXXXXX — $case_name",  
    legend = :topright  
)
```

— Сохранение графика —

```
savefig(plt, plotsdir(script_name, "$(image_name).png"))
```

— Сохранение таблицы —

```
CSV.write(datadir(script_name, "table_$(case_name).csv"), df)
```

— Сохранение данных в JLD2 —

```
@save datadir(script_name, "data_$(case_name).jld2") df p u0 ts

return (
    sol = sol,
    df = df,
    plt = plt
)
end
```

5.7 Запуск первой модели

```
res_1 = run_case(
    "case_1";
    model! = model_case_1!,
    u0 = u0,
    tspan = tspan_1,
    p = p1,
    nt = 500,
    image_name = "04"
)
```

5.8 Запуск второй модели

```
res_2 = run_case(  
    "case_2";  
    model! = model_case_2!,  
    u0 = u0,  
    tspan = tspan_2,  
    p = p2,  
    nt = 500,  
    image_name = "05"  
)
```

5.9 Запуск третьей модели

```
res_3 = run_case(  
    "case_3";  
    model! = model_case_3!,  
    u0 = u0,  
    tspan = tspan_3,  
    p = p3,  
    nt = 500,  
    image_name = "06"  
)
```

5.10 Вывод первых строк таблиц

```
println("Таблица 5: Первые 5 строк таблицы")
println(first(res_1.df, 5))

println()
println("Таблица 5: Первые 5 строк таблицы")
println(first(res_2.df, 5))

println()
println("Таблица 5: Первые 5 строк таблицы")
println(first(res_3.df, 5))
```

5.11 Показать график в интерактивной среде

```
res_1.plt
```

6 Параметрическое исследование трёх дифференциальных моделей

6.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
using CSV
using Statistics
```

6.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE)).ext
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

6.3 Определение моделей

Первая модель: $dn/dt = (a + bn)(N - n)$

```
function model_case_1!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end
```

Вторая модель: $dn/dt = (a + bn)(N - n)$

```
function model_case_2!(dy, y, p, t)
    dy[1] = (p.a + p.b * y[1]) * (p.N - y[1])
end
```

Третья модель: $dn/dt = (a\cos(t) + b\cos(2t)n)(N - n)$

```
function model_case_3!(dy, y, p, t)
    dy[1] = (p.a * cos(t) + p.b * cos(2 * t) * y[1]) * (p.N - y[1])
end
```

6.4 Выбор модели по типу эксперимента

```
function choose_model(model_type)
    if model_type == :case1
        return model_case_1!
    elseif model_type == :case2
        return model_case_2!
    elseif model_type == :case3
        return model_case_3!
    else
```

```

        error("XXXXXXXXXX XX XXXXX: $model_type")
    end
end

```

6.5 Правая часть уравнения для анализа производной

```

function rhs_value(model_type, n, p, t)
    if model_type == :case1
        return (p.a + p.b * n) * (p.N - n)
    elseif model_type == :case2
        return (p.a + p.b * n) * (p.N - n)
    elseif model_type == :case3
        return (p.a * cos(t) + p.b * cos(2 * t) * n) * (p.N - n)
    else
        error("XXXXXXXXXX XX XXXXX: $model_type")
    end
end

```

6.6 Базовые параметры первой модели

```

base_params1 = Dict(
    :N => 609.0,
    :n0 => 4.0,

    :tspan => (0.0, 10.0),

```

```

:nt => 500,

:model_type => :case1,

:a => 0.54,
:b => 0.00016,

:solver => Tsit5(),
:experiment_name => "base_experiment_case1"
)

```

6.7 Базовые параметры второй модели

```

base_params2 = Dict(
  :N => 609.0,
  :n0 => 4.0,

  :tspan => (0.0, 0.04),
  :nt => 500,

  :model_type => :case2,

  :a => 0.000021,
  :b => 0.38,

  :solver => Tsit5(),
  :experiment_name => "base_experiment_case2"
)

```

6.8 Базовые параметры третьей модели

```
base_params3 = Dict(
    :N => 609.0,
    :n0 => 4.0,

    :tspan => (0.0, 0.15),
    :nt => 500,

    :model_type => :case3,

    :a => 0.2,
    :b => 0.2,

    :solver => Tsit5(),
    :experiment_name => "base_experiment_case3"
)

base_experiments = [base_params1, base_params2, base_params3]

println("XXXXXXXX XXXXXXXXXXX XXXXXXXXXXXXXXXX:")
for params in base_experiments
    println("\nmodel_type = ", params[:model_type])
    println(" N = ", params[:N])
    println(" n0 = ", params[:n0])
    println(" tspan = ", params[:tspan])
```

```
println(" nt = ", params[:nt])
println(" a = ", params[:a])
println(" b = ", params[:b])
end
```

6.9 Функция для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    N = params[:N]
    n0 = params[:n0]

    tspan = params[:tspan]
    nt = params[:nt]

    model_type = params[:model_type]
    model! = choose_model(model_type)

    a = params[:a]
    b = params[:b]

    solver = params[:solver]

    u0 = [n0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    p = (a = a, b = b, N = N)

    prob = ODEProblem(model!, u0, tspan, p)
```

```

sol = solve(prob, solver; saveat = tgrid)

t_vals = sol.t
n_vals = [u[1] for u in sol.u]
dn_vals = [rhs_value(model_type, n_vals[i], p, t_vals[i]) for i in 1:length(n_vals)]

```

— Мини-анализ —

```

n_initial = n_vals[1]
n_final = n_vals[end]

n_max = maximum(n_vals)
n_min = minimum(n_vals)
n_mean = mean(n_vals)

dn_final = dn_vals[end]
dn_max = maximum(dn_vals)
dn_min = minimum(dn_vals)
dn_mean = mean(dn_vals)

growth_abs = n_final - n_initial
growth_rel = growth_abs / n_initial

saturation_final = n_final / N
saturation_max = n_max / N

idx_n_max = argmax(n_vals)
t_at_n_max = t_vals[idx_n_max]

```

```

idx_90 = findfirst(x -> x >= 0.9 * N, n_vals)
t_reach_90 = isnothing(idx_90) ? missing : t_vals[idx_90]

idx_95 = findfirst(x -> x >= 0.95 * N, n_vals)
t_reach_95 = isnothing(idx_95) ? missing : t_vals[idx_95]

return Dict(
    "solution" => sol,
    "t_points" => t_vals,

    "n_values" => n_vals,
    "dn_values" => dn_vals,

    "n_initial" => n_initial,
    "n_final" => n_final,

    "n_max" => n_max,
    "n_min" => n_min,
    "n_mean" => n_mean,

    "dn_final" => dn_final,
    "dn_max" => dn_max,
    "dn_min" => dn_min,
    "dn_mean" => dn_mean,

    "growth_abs" => growth_abs,
    "growth_rel" => growth_rel,

```

```

        "saturation_final" => saturation_final,
        "saturation_max" => saturation_max,

        "t_at_n_max" => t_at_n_max,
        "t_reach_90" => t_reach_90,
        "t_reach_95" => t_reach_95,

        "parameters" => params
    )
end

```

6.10 Функция печати результатов одного эксперимента

```

function print_experiment_summary(data, path)
    println(" n_initial: ", data["n_initial"])
    println(" n_final: ", data["n_final"])
    println(" n_min: ", data["n_min"])
    println(" n_max: ", data["n_max"])
    println(" n_mean: ", round(data["n_mean"]; digits = 4))

    println(" dn_final: ", data["dn_final"])
    println(" dn_min: ", data["dn_min"])
    println(" dn_max: ", data["dn_max"])
    println(" dn_mean: ", round(data["dn_mean"]; digits = 4))

    println(" growth_abs: ", data["growth_abs"])

```

```

println(" growth_rel: ", round(data["growth_rel"]; digits = 4))

println(" saturation_final: ", round(data["saturation_final"]; digits = 4))
println(" saturation_max: ", round(data["saturation_max"]; digits = 4))

println(" t_at_n_max: ", data["t_at_n_max"])
println(" t_reach_90: ", data["t_reach_90"])
println(" t_reach_95: ", data["t_reach_95"])

println(" XXXX XXXXXXXXXXXX: ", path)
end

```

6.11 Функция построения графика $n(t)$

```

function plot_time_series(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["t_points"],
        data["n_values"],
        xlabel = "t",
        ylabel = "n(t)",
        label = "n(t)",
        title = "XXXXXX n(t): model_type=$model_type",
        lw = 2,
        legend = :bottomright,
        grid = true
    )

```

```

hline!(
    plt,
    [params[:N]],
    label = "N",
    lw = 2,
    linestyle = :dash
)

return plt
end

```

6.12 Функция построения графика производной dn/dt

```

function plot_derivative_series(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["t_points"],
        data["dn_values"],
        xlabel = "t",
        ylabel = "dn/dt",
        label = "dn/dt",
        title = "XXXXXXXXXX XXXXXXXXXXXX: model_type=$model_type",
        lw = 2,
        legend = :topright,
    )
end

```

```

        grid = true
    )

    return plt
end

```

6.13 Функция построения фазовой траектории dn/dt от n

```

function plot_phase_n_dn(data, params)
    model_type = params[:model_type]

    plt = plot(
        data["n_values"],
        data["dn_values"],
        xlabel = "n",
        ylabel = "dn/dt",
        label = "XXXXXXXXXXXXXXXXXXXX",
        title = "XXXXXXXXXXXXXXXXXXXX dn/dt(n): model_type=$model_type",
        lw = 2,
        legend = :topright,
        grid = true
    )

    return plt
end

```

6.14 Запуск базовых экспериментов

```
base_results = Dict()

for params in base_experiments
    model_type = params[:model_type]

    println("\n" * "="^60)
    println("XXXXXXXXXXXXXXXX: model_type=$model_type")
    println("="^60)

    data, path = produce_or_load(
        datadir(script_name, "single"),
        params,
        run_single_experiment;
        prefix = "model",
        tag = false,
        verbose = true
    )

    base_results[model_type] = Dict(
        "data" => data,
        "path" => path
    )

    print_experiment_summary(data, path)

    plt_time = plot_time_series(data, params)
```

```

    savefig(plt_time, plotsdir(script_name, "single_experiment_$(model_type).png")
    plt_derivative = plot_derivative_series(data, params)
    savefig(plt_derivative, plotsdir(script_name, "single_experiment_$(model_type).png")

    plt_phase = plot_phase_n_dn(data, params)
    savefig(plt_phase, plotsdir(script_name, "single_experiment_$(model_type).png")

    df_base = DataFrame(
        t = data["t_points"],
        n = data["n_values"],
        dn = data["dn_values"],
        model_type = fill(string(model_type), length(data["t_points"])),
        a = fill(params[:a], length(data["t_points"])),
        b = fill(params[:b], length(data["t_points"])),
        N = fill(params[:N], length(data["t_points"]))
    )

    CSV.write(datadir(script_name, "table_single_$(model_type).csv"), df_base, "csv")
end

```

6.15 Параметрическое сканирование

Сетка параметров для первой модели

```

param_grid_case1 = Dict{
    :N => [609.0],
    :n0 => [4.0],
}

```

```

:tspan => [(0.0, 10.0)],
:nt => [500],

:model_type => [:case1],

:a => [0.35, 0.54, 0.70],
:b => [0.00008, 0.00016, 0.00024],

:solver => [Tsit5()],
:experiment_name => ["parametric_scan_case1"]
)

```

Сетка параметров для второй модели

```

param_grid_case2 = Dict(
    :N => [609.0],
    :n0 => [4.0],

    :tspan => [(0.0, 0.04)],
    :nt => [500],

    :model_type => [:case2],

    :a => [0.00001, 0.000021, 0.00004],
    :b => [0.25, 0.38, 0.50],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan_case2"]
)

```

Сетка параметров для третьей модели

```
param_grid_case3 = Dict(
    :N => [609.0],
    :n0 => [4.0],

    :tspan => [(0.0, 0.15)],
    :nt => [500],

    :model_type => [:case3],

    :a => [0.10, 0.20, 0.30],
    :b => [0.10, 0.20, 0.30],

    :solver => [Tsit5()],
    :experiment_name => ["parametric_scan_case3"]
)

all_params = vcat(
    dict_list(param_grid_case1),
    dict_list(param_grid_case2),
    dict_list(param_grid_case3)
)

println("\n" * "="^60)
println("XXXXXXXXXXXXXXXXXXXX XXXXXXXXXXXXXXX")
println("XXXXXX XXXXXXXXXXXXXXX XXXXXXXXXXXXXXX: ", length(all_params))
println("="^60)
```

6.16 Запуск всех экспериментов

```
all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println(
        "XXXXXXXXXX: $i/${length(all_params)} | ",
        "model_type=$(params[:model_type]) | ",
        "a=$(params[:a]) | b=$(params[:b])"
    )

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    result_summary = (
        model_type = string(params[:model_type]),

        N = params[:N],
        n0 = params[:n0],

        t_start = params[:tspan][1],
```

```

t_end = params[:tspan][2],
nt = params[:nt],

a = params[:a],
b = params[:b],

n_initial = data["n_initial"],
n_final = data["n_final"],

n_min = data["n_min"],
n_max = data["n_max"],
n_mean = data["n_mean"],

dn_final = data["dn_final"],
dn_min = data["dn_min"],
dn_max = data["dn_max"],
dn_mean = data["dn_mean"],

growth_abs = data["growth_abs"],
growth_rel = data["growth_rel"],

saturation_final = data["saturation_final"],
saturation_max = data["saturation_max"],

t_at_n_max = data["t_at_n_max"],
t_reach_90 = data["t_reach_90"],
t_reach_95 = data["t_reach_95"],

```

```

        filepath = path
    )

    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        n = data["n_values"],
        dn = data["dn_values"],

        model_type = fill(string(params[:model_type]), length(data[

        N = fill(params[:N], length(data["t_points"])),
        n0 = fill(params[:n0], length(data["t_points"])),

        a = fill(params[:a], length(data["t_points"])),
        b = fill(params[:b], length(data["t_points"])),

        t_start = fill(params[:tspan][1], length(data["t_points"])),
        t_end = fill(params[:tspan][2], length(data["t_points"]))
    )

    push!(all_dfs, df)
end

```

6.17 Анализ результатов сканирования

```
results_df = DataFrame(all_results)
full_timeseries_df = vcat(all_dfs...)

println("\nXXXXXXXX XXXXXX XXXXXXXXXXXX:")
println(first(results_df, 10))

CSV.write(datadir(script_name, "results_summary.csv"), results_df)
CSV.write(datadir(script_name, "timeseries_full.csv"), full_timeser
```

6.18 Сравнительные графики $n(t)$ для каждой модели

```
model_types = unique(results_df.model_type)

for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) ==

    plt = plot(size = (950, 520), dpi = 150)

    for params in params_subset
        data, _ = produce_or_load(
            datadir(script_name, "parametric_scan"),
            params,
            run_single_experiment;
            prefix = "scan",
            tag = false,
            verbose = false
```

```

    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["t_points"],
        data["n_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    plt,
    xlabel = "t",
    ylabel = "n(t)",
    title = "XXXXXXXXXXXX: XXXXXXXXXX n(t), model_type=$model_type_s",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_n_comparison")
end

```

6.19 Сравнительные графики dn/dt для каждой модели

```
for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) ==

plt = plot(size = (950, 520), dpi = 150)

for params in params_subset
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["t_points"],
        data["dn_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
```

```

end

plot!(
    plt,
    xlabel = "t",
    ylabel = "dn/dt",
    title = "XXXXXXXXXXXX: XXXXXXXX XXXXXXXX, model_type=$model_type",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_dn_comparis
end

```

6.20 Сравнительные фазовые траектории $dn/dt(n)$

```

for model_type_string in model_types
    params_subset = filter(params -> string(params[:model_type]) ==

    plt = plot(size = (950, 520), dpi = 150)

    for params in params_subset
        data, _ = produce_or_load(
            datadir(script_name, "parametric_scan"),
            params,
            run_single_experiment;
            prefix = "scan",
            tag = false,

```

```

        verbose = false
    )

    label_text = "a=$(params[:a]), b=$(params[:b])"

    plot!(
        plt,
        data["n_values"],
        data["dn_values"],
        label = label_text,
        lw = 2,
        alpha = 0.8
    )
end

plot!(
    plt,
    xlabel = "n",
    ylabel = "dn/dt",
    title = "XXXXXXXXXXXX: XXXXXXX XXXXXXXX dn/dt(n), model_type=$model_type",
    legend = :outright,
    grid = true
)

savefig(plt, plotsdir(script_name, "parametric_scan_phase_n_dn_"),
end

```

6.21 График зависимости n_{final} от параметра a

```
p_a_final = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_final,
            sub.a,
            sub.n_final,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\text{final}}$  vs  $a$ "
        )
    end
end

plot!(
    p_a_final,
    xlabel = "a",
    ylabel = " $n_{\text{final}}$ ",
    title = "Зависимость  $n_{\text{final}}$  от параметра  $a$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_a_final, plotsdir(script_name, "n_final_vs_a.png"))
```

6.22 График зависимости n_{final} от параметра b

```
p_b_final = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_final,
            sub.b,
            sub.n_final,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\text{final}}$  vs  $b$ "
        )
    end
end

plot!(
    p_b_final,
    xlabel = "b",
    ylabel = " $n_{\text{final}}$ ",
    title = "Scatter plot of  $n_{\text{final}}$  vs  $b$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_b_final, plotsdir(script_name, "n_final_vs_b.png"))
```

6.23 График зависимости $n_{\max}$ от параметра a

```
p_a_max = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_max,
            sub.a,
            sub.n_max,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\max}$  vs  $a$ "
        )
    end
end

plot!(
    p_a_max,
    xlabel = "a",
    ylabel = " $n_{\max}$ ",
    title = "Зависимость  $n_{\max}$  от параметра  $a$ ",
    legend = :bottomright,
    grid = true
)

savefig(p_a_max, plotsdir(script_name, "n_max_vs_a.png"))
```

6.24 График зависимости $n_{\max}$ от параметра b

```
p_b_max = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_max,
            sub.b,
            sub.n_max,
            seriestype = :scatter,
            label = "$model_type_string:  $n_{\max}$  vs  $b$ "
        )
    end
end

plot!(
    p_b_max,
    xlabel = "b",
    ylabel = " $n_{\max}$ ",
    title = "Зависимость  $n_{\max}$  от параметра  $b$  для различных моделей",
    legend = :bottomright,
    grid = true
)

savefig(p_b_max, plotsdir(script_name, "n_max_vs_b.png"))
```

6.25 График зависимости `saturation_final` от параметра `a`

```
p_a_saturation = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_a_saturation,
            sub.a,
            sub.saturation_final,
            seriestype = :scatter,
            label = "$model_type_string: saturation_final vs a"
        )
    end
end

plot!(
    p_a_saturation,
    xlabel = "a",
    ylabel = "n_final / N",
    title = "Среднее значение n_final / N в зависимости от параметра a",
    legend = :bottomright,
    grid = true
)
```

```
savefig(p_a_saturation, plotsdir(script_name, "saturation_final_vs_
```

6.26 График зависимости `saturation_final` от параметра `b`

```
p_b_saturation = plot(size = (900, 520), dpi = 150)

for model_type_string in model_types
    sub = results_df[results_df.model_type .== model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_b_saturation,
            sub.b,
            sub.saturation_final,
            seriestype = :scatter,
            label = "$model_type_string: saturation_final vs b"
        )
    end
end

plot!(
    p_b_saturation,
    xlabel = "b",
    ylabel = "n_final / N",
    title = "Среднее значение n_final / N в зависимости от параметра b",
    legend = :bottomright,
```

```

        grid = true
    )

    savefig(p_b_saturation, plotsdir(script_name, "saturation_final_vs_

```

6.27 Бенчмаркинг

```

println("\n" * "="^60)
println("XXXXXXXXXX  XX  XXXXXX  XXXXXXXXXXXX")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        N = params[:N]
        n0 = params[:n0]

        u0 = [n0]
        p = (a = params[:a], b = params[:b], N = N)

        model! = choose_model(params[:model_type])

        prob = ODEProblem(model!, u0, params[:tspan], p)

        return solve(
            prob,

```

```

        params[:solver];
        saveat = LinRange(params[:tspan][1], params[:tspan][2],
    )
end

println(
    "\nXXXXXXXXXX XXXX model_type=$(params[:model_type]), ",
    "a=$(params[:a]), b=$(params[:b]):"
)

bmark = @benchmark $benchmark_run() samples = 80 evals = 1
tsec = median(bmark).time / 1e9

println(" XXXXXXXXXXXX XXXXXX: ", round(tsec; digits = 6), " XXXX")

push!(
    benchmark_results,
    (
        model_type = string(params[:model_type]),
        a = params[:a],
        b = params[:b],
        time = tsec
    )
)
end

bench_df = DataFrame(benchmark_results)
CSV.write(datadir(script_name, "benchmark_results.csv"), bench_df)

```

6.28 График времени вычисления от параметра a

```
p_time_a = plot(size = (900, 520), dpi = 150)

for model_type_string in unique(bench_df.model_type)
    sub = bench_df[bench_df.model_type == model_type_string, :]

    if nrow(sub) > 0
        plot!(
            p_time_a,
            sub.a,
            sub.time,
            seriestype = :scatter,
            label = "$model_type_string:  $\frac{1}{a}$  a"
        )
    end
end

plot!(
    p_time_a,
    xlabel = "a",
    ylabel = " $\frac{1}{a}$   $\frac{1}{a}$ ",
    title = " $\frac{1}{a}$   $\frac{1}{a}$  ODE  $\frac{1}{a}$   $\frac{1}{a}$  a",
    legend = :topright,
    grid = true
)

savefig(p_time_a, plotsdir(script_name, "computation_time_vs_a.png"))
```

6.29 График времени вычисления от параметра b

```
p_time_b = plot(size = (900, 520), dpi = 150)

for model_type_string in unique(bench_df.model_type)
  sub = bench_df[bench_df.model_type == model_type_string, :]

  if nrow(sub) > 0
    plot!(
      p_time_b,
      sub.b,
      sub.time,
      seriestype = :scatter,
      label = "$model_type_string:  $\frac{b}{t}$  b"
    )
  end
end

plot!(
  p_time_b,
  xlabel = "b",
  ylabel = " $\frac{b}{t}$   $\frac{b}{t}$ ",
  title = " $\frac{b}{t}$   $\frac{b}{t}$  ODE  $\frac{b}{t}$   $\frac{b}{t}$  b",
  legend = :topright,
  grid = true
)

savefig(p_time_b, plotsdir(script_name, "computation_time_vs_b.png"))
```

6.30 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_experiments base_results

@save datadir(script_name, "all_plots.jld2") p_a_final p_b_final p_c_final

println("\n" * "="^60)
println("XXXXXXXXXXXXXXXX XXXXXX XXXXXXXXXXXX")
println("="^60)

println("\nXXXXXXXXXXXXXXXX XXXXXXXXXXXX X:")
println(" • data/${script_name}/single/ - XXXXXXXX XXXXXXXXXXXX")
println(" • data/${script_name}/parametric_scan/ - XXXXXXXXXXXXXXXXXXXX XXXXXXXX")
println(" • data/${script_name}/table_single_case1.csv - XXXXXXXX XXX XXXXXXXX")
println(" • data/${script_name}/table_single_case2.csv - XXXXXXXX XXX XXXXXXXX")
println(" • data/${script_name}/table_single_case3.csv - XXXXXXXX XXX XXXXXXXX")
println(" • data/${script_name}/results_summary.csv - XXXXXXXX XXXXXXXXXXXX")
println(" • data/${script_name}/timeseries_full.csv - XXXXXXXX XXXXXXXXXXXX")
println(" • data/${script_name}/benchmark_results.csv - XXXXXXXXXXXX XXXXXXXX")
println(" • data/${script_name}/all_results.jld2 - XXXXXXXX XXXXXXXX")
println(" • data/${script_name}/all_plots.jld2 - XXXXXXXX XXXXXXXXXXXX")
println(" • plots/${script_name}/ - XXX XXXXXXXX")
```

6.31 Базовые эксперименты

6.31.1 Первая модель ($\text{model_type} = \text{case1}$)

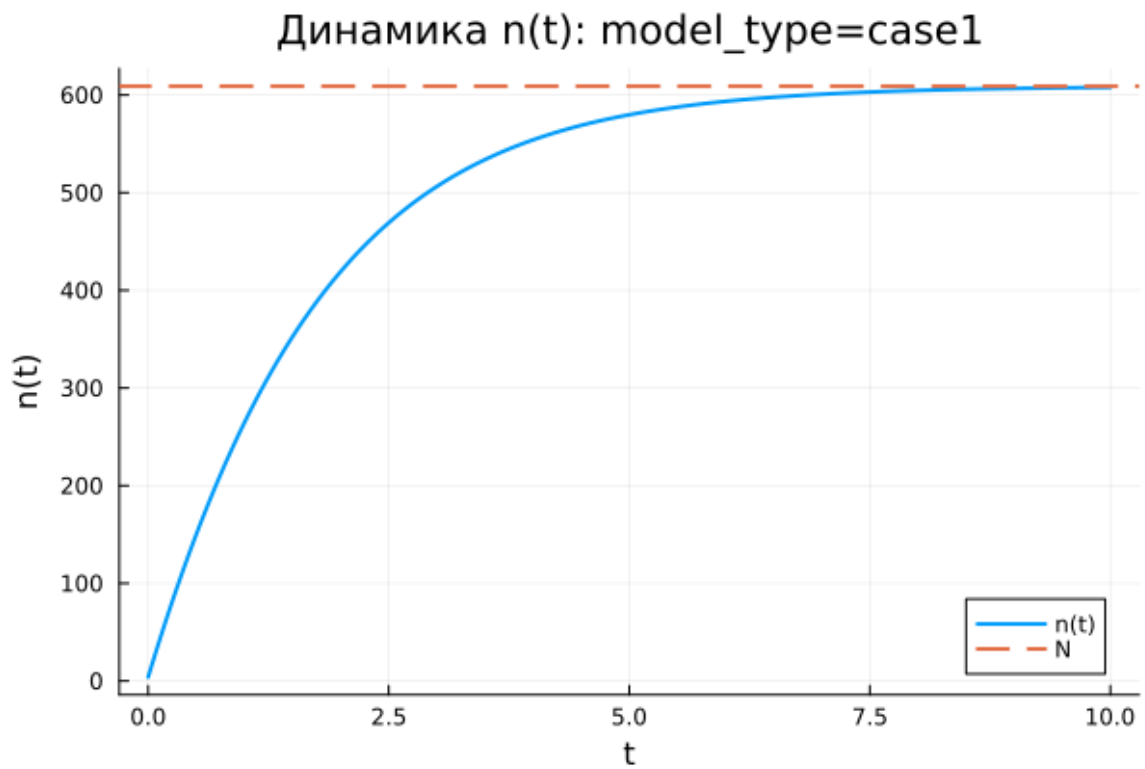


Рисунок 6.1: Первая модель: зависимость $n(t)$

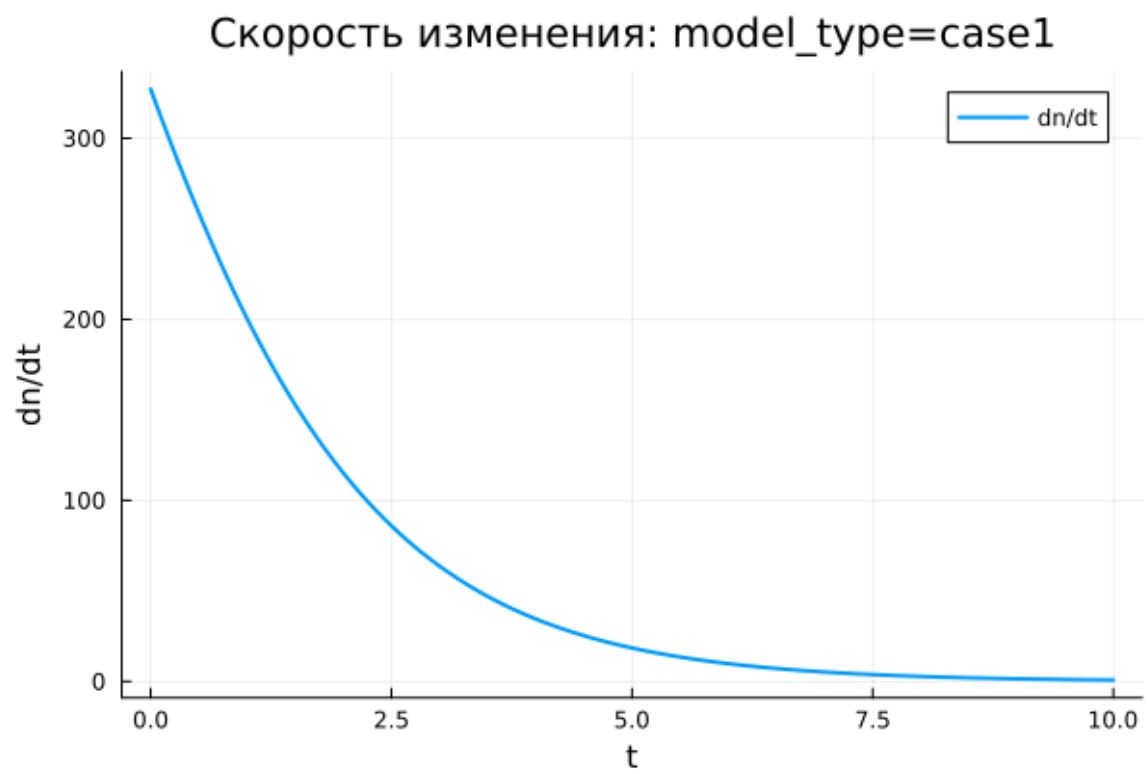


Рисунок 6.2: Первая модель: скорость изменения

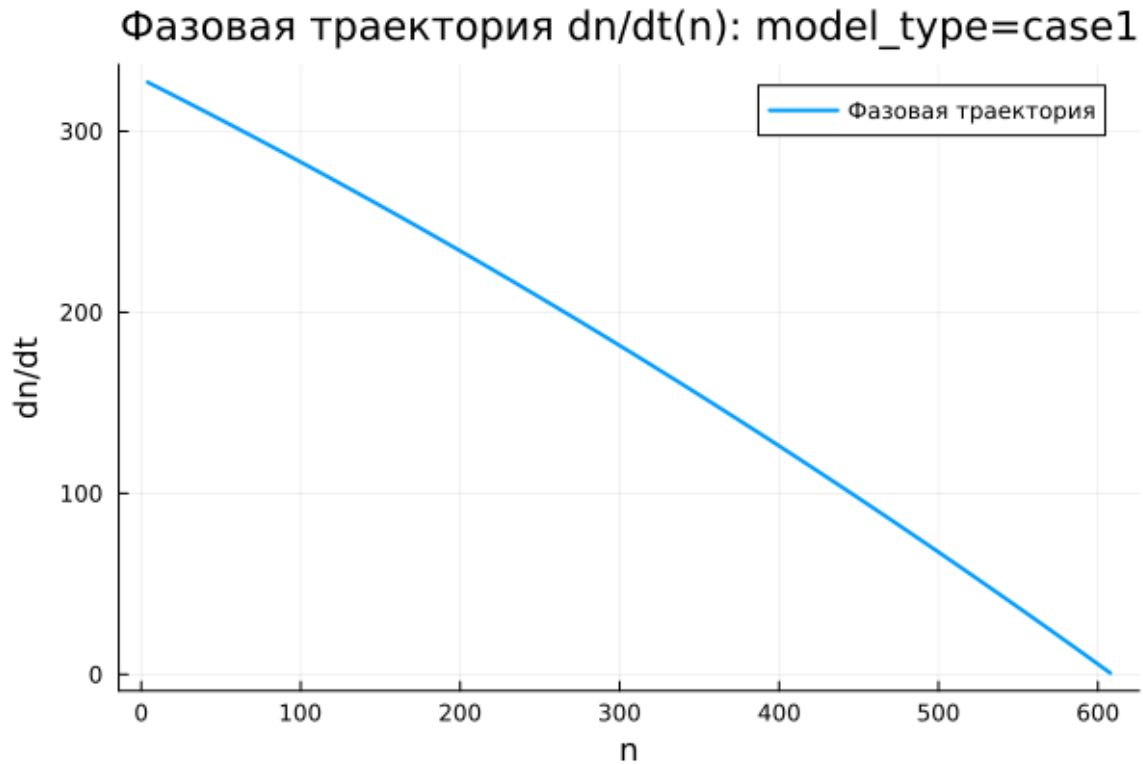


Рисунок 6.3: Первая модель: фазовая траектория

Для первой модели наблюдается монотонный рост величины $n(t)$. В начальный момент значение n мало по сравнению с предельным уровнем $N = 609$, поэтому процесс развивается достаточно быстро. Затем по мере приближения $n(t)$ к N рост постепенно замедляется.

На графике динамики видно, что к концу расчётного интервала $t \in [0; 10]$ решение почти достигает уровня насыщения. Горизонтальная пунктирная линия соответствует значению N , к которому стремится численное решение. Переменная $n(t)$ не превышает предельный уровень, а плавно приближается к нему снизу.

График скорости изменения показывает, что максимальная скорость наблюдается в начальный момент времени. Далее dn/dt монотонно убывает и стремится к нулю. Это означает, что основной прирост происходит в начале процесса, после чего система постепенно переходит к состоянию насыщения.

Фазовая траектория $dn/dt(n)$ подтверждает данный вывод. При малых значениях

n скорость изменения максимальна, а при приближении к N она уменьшается почти до нуля. Траектория имеет убывающий вид, поэтому модель описывает процесс с быстрым начальным ростом и дальнейшим замедлением.

Первая модель демонстрирует устойчивое стремление к предельному уровню N . Состояние $n = N$ выступает равновесным: при достижении насыщения множитель $(N - n)$ обращает правую часть уравнения в ноль.

6.31.2 Вторая модель (model_type = case2)

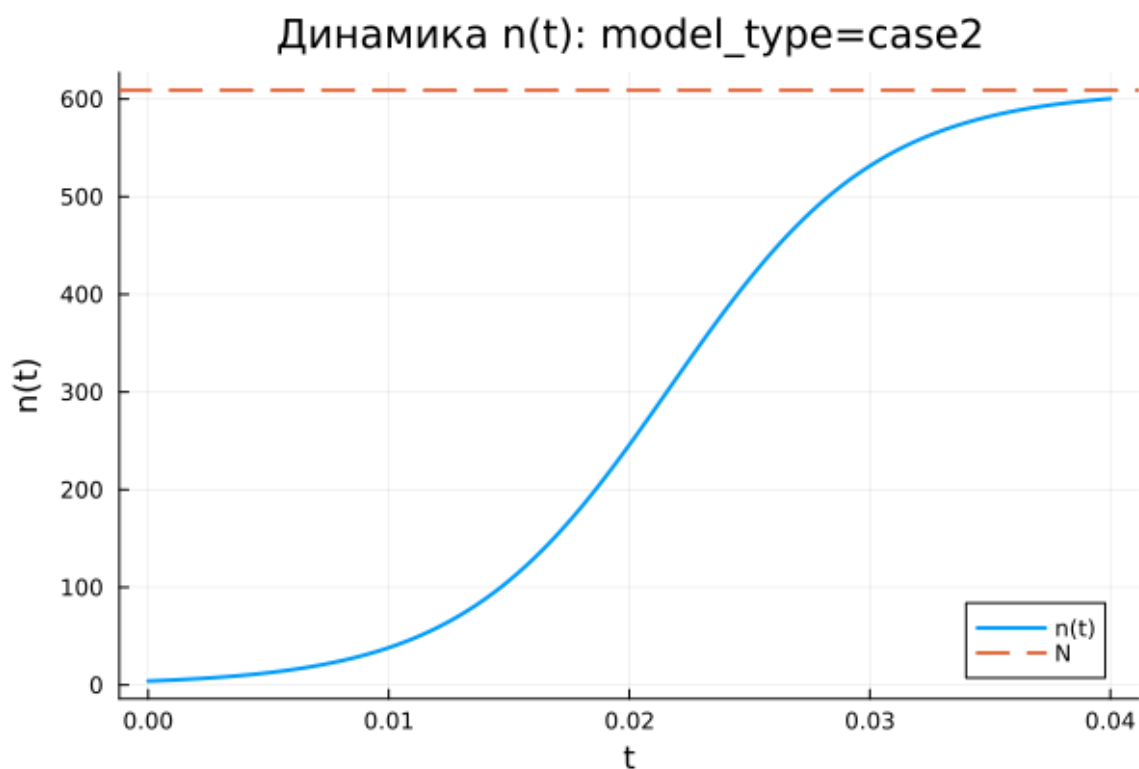


Рисунок 6.4: Вторая модель: зависимость $n(t)$

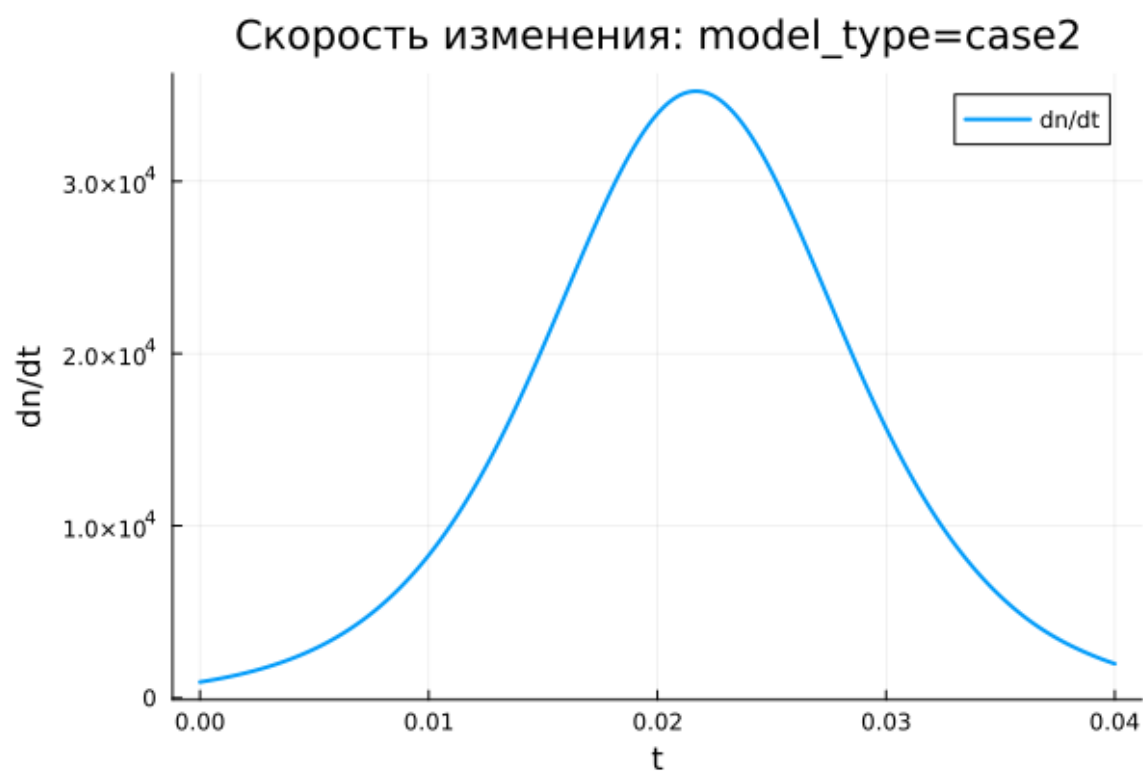


Рисунок 6.5: Вторая модель: скорость изменения

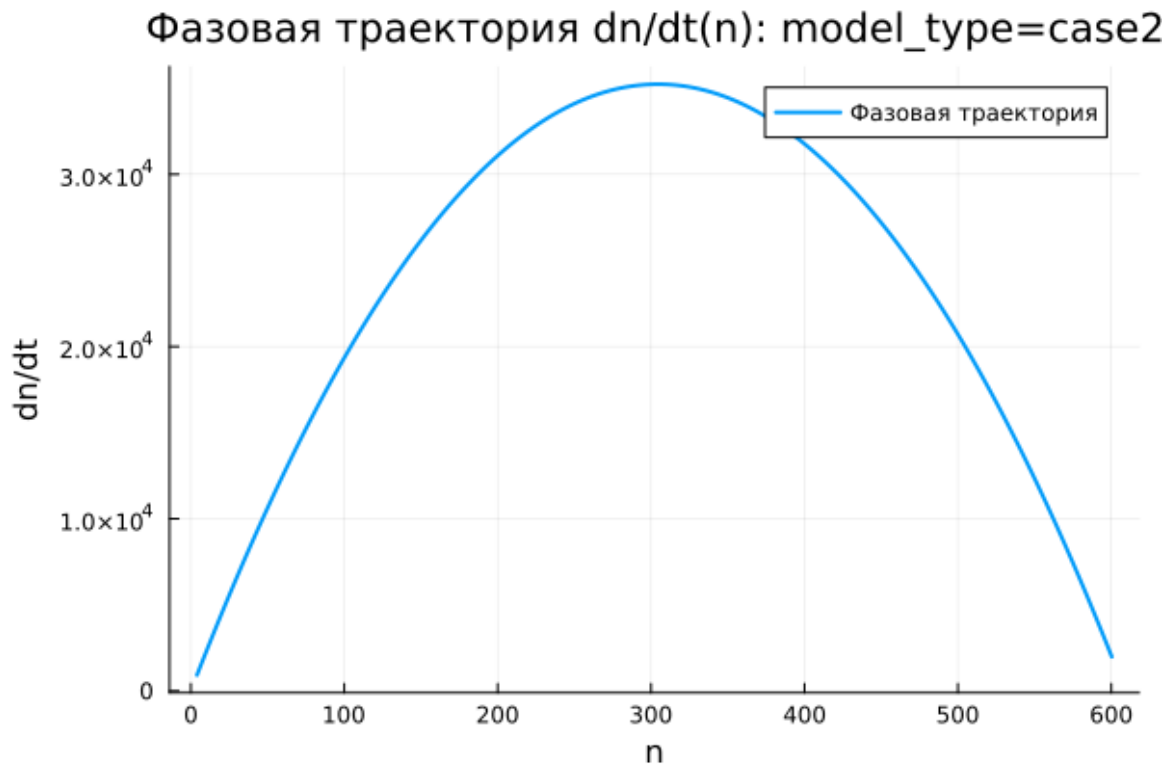


Рисунок 6.6: Вторая модель: фазовая траектория

Во второй модели величина $n(t)$ возрастает значительно быстрее, чем в первой. Расчётный интервал очень мал: $t \in [0; 0,04]$, однако за это время решение почти достигает уровня $N = 609$. Это связано с большим значением коэффициента b , который усиливает вклад слагаемого bn .

График $n(t)$ имеет выраженную S-образную форму. В начале рост сравнительно медленный, поскольку значение n ещё невелико. Затем скорость резко увеличивается, после чего снова снижается при приближении к предельному уровню N . В результате решение быстро выходит на насыщение.

График dn/dt показывает наличие одного выраженного максимума. Сначала скорость изменения возрастает, достигает пика примерно в середине активной фазы процесса, а затем убывает почти до нуля. Это означает, что модель описывает режим ускоренного роста с последующим торможением из-за ограничения $(N - n)$.

Фазовая траектория $dn/dt(n)$ имеет форму параболы. При малых значениях n

скорость изменения невелика, затем она увеличивается и достигает максимума примерно при $n \approx 300$, после чего снижается при движении к N . Это отражает конкуренцию двух факторов: самоускорения через множитель bn и ограничения через множитель $(N - n)$.

Вторая модель описывает более резкий переход к насыщению. В отличие от первой модели, максимальная скорость достигается не в начале, а на внутреннем участке траектории, где значение n уже достаточно велико, но запас до N ещё остаётся существенным.

6.31.3 Третья модель (model_type = case3)

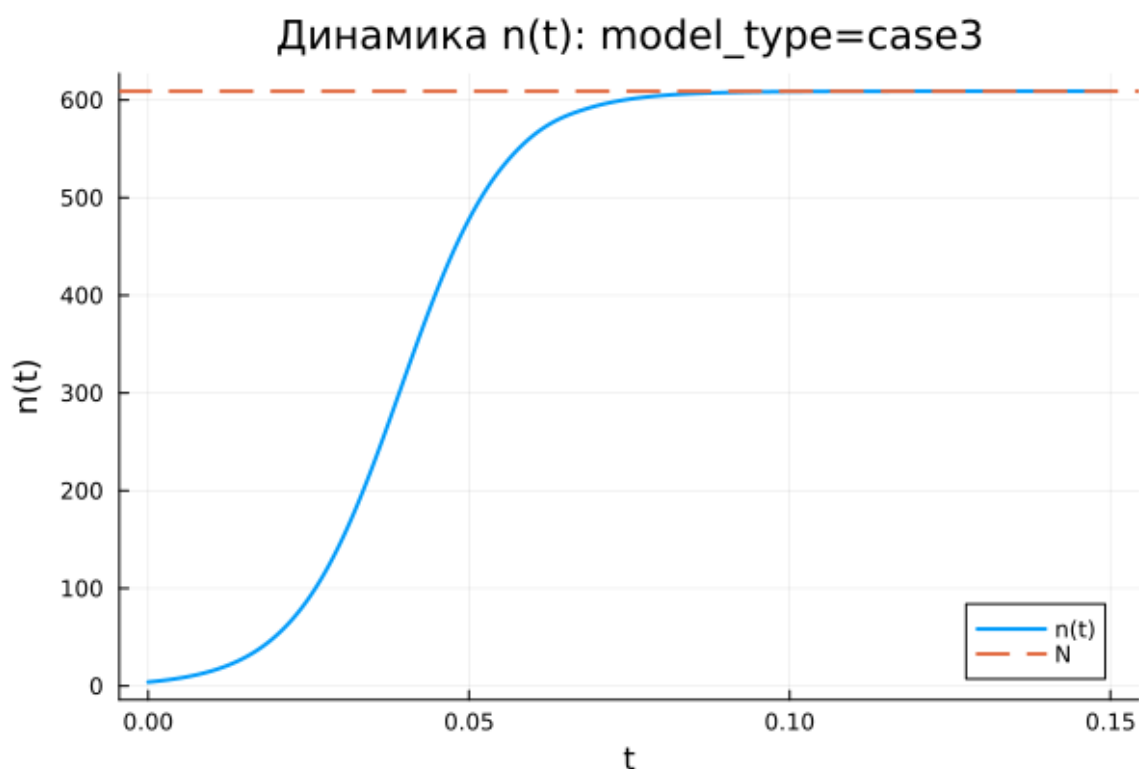


Рисунок 6.7: Третья модель: зависимость $n(t)$

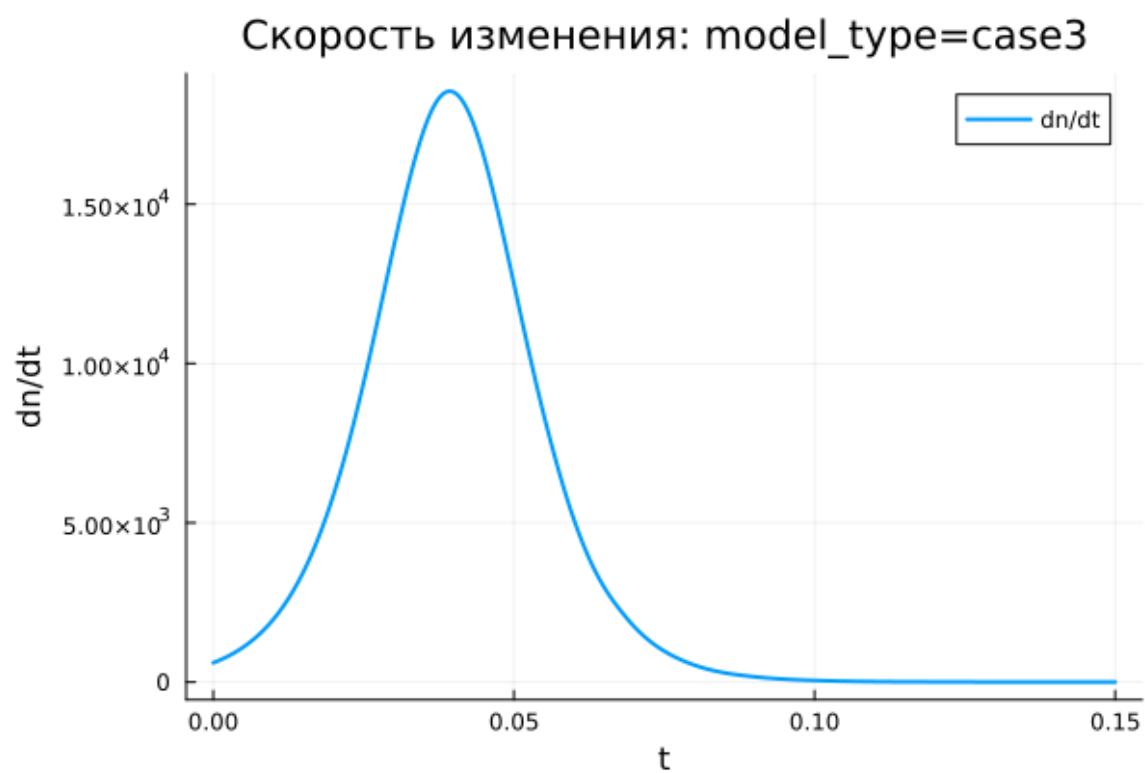


Рисунок 6.8: Третья модель: скорость изменения

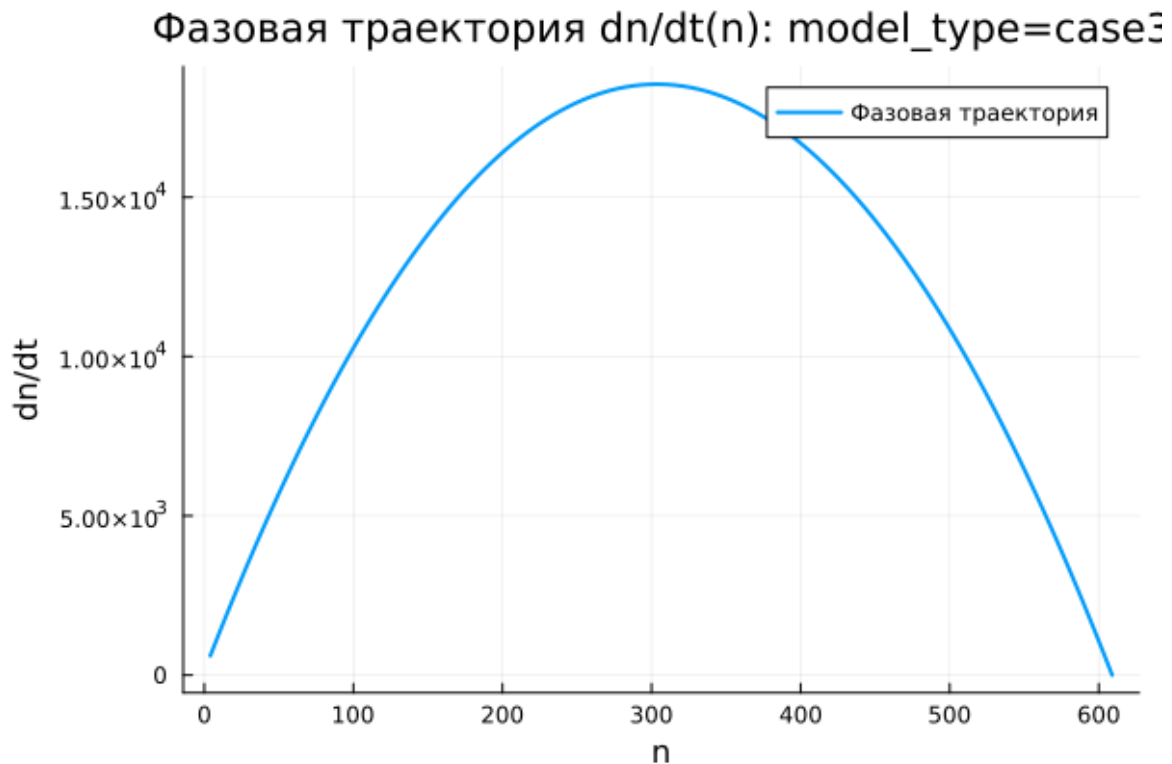


Рисунок 6.9: Третья модель: фазовая траектория

В третьей модели величина $n(t)$ монотонно возрастает и быстро приближается к уровню $N = 609$. На графике видно, что основная часть роста происходит на начальном участке интервала $t \in [0; 0,15]$. После этого кривая почти сливается с горизонтальным уровнем насыщения.

Особенность третьей модели состоит в наличии периодических множителей $\cos(t)$ и $\cos(2t)$. На выбранном коротком интервале времени эти функции остаются положительными и близкими к единице, поэтому они не вызывают колебаний решения, а лишь изменяют интенсивность роста.

График скорости dn/dt имеет один ярко выраженный пик. В начале процесса скорость возрастает, затем достигает максимального значения, после чего быстро убывает к нулю. После выхода $n(t)$ на уровень насыщения дальнейшее изменение практически прекращается.

Фазовая траектория $dn/dt(n)$ по форме близка к параболической. Скорость

изменения мала при небольших значениях n , затем возрастает и достигает максимума примерно в средней части диапазона. При приближении к N множитель $(N - n)$ уменьшает скорость до нуля.

Третья модель демонстрирует насыщаемый рост с переменными во времени коэффициентами. На данном интервале периодические множители не меняют общий характер процесса: решение остаётся монотонным, достигает предельного уровня и переходит в состояние равновесия.

6.32 Сравнение базовых экспериментов

Во всех трёх моделях величина $n(t)$ стремится к одному и тому же предельному уровню $N = 609$. Это связано с наличием множителя $(N - n)$, который уменьшает скорость роста при приближении к насыщению и обращает её в ноль при $n = N$.

Первая модель отличается быстрым стартом и дальнейшим плавным замедлением. Максимальная скорость наблюдается в начале процесса, а фазовая траектория имеет убывающий характер.

Вторая модель показывает наиболее резкий S-образный рост. Скорость сначала увеличивается, затем достигает максимума и убывает. Фазовая траектория имеет выраженную параболическую форму, что указывает на наличие внутренней точки максимального роста.

Третья модель по характеру близка ко второй, но включает зависимость коэффициентов от времени. На рассматриваемом интервале эта зависимость не приводит к колебательному режиму, поскольку значения $\cos(t)$ и $\cos(2t)$ остаются положительными.

Все три модели приводят систему к насыщению, но делают это с разной скоростью и с разным характером изменения производной dn/dt .

6.33 Параметрическое сканирование

6.33.1 Зависимость итогового значения n от параметра a

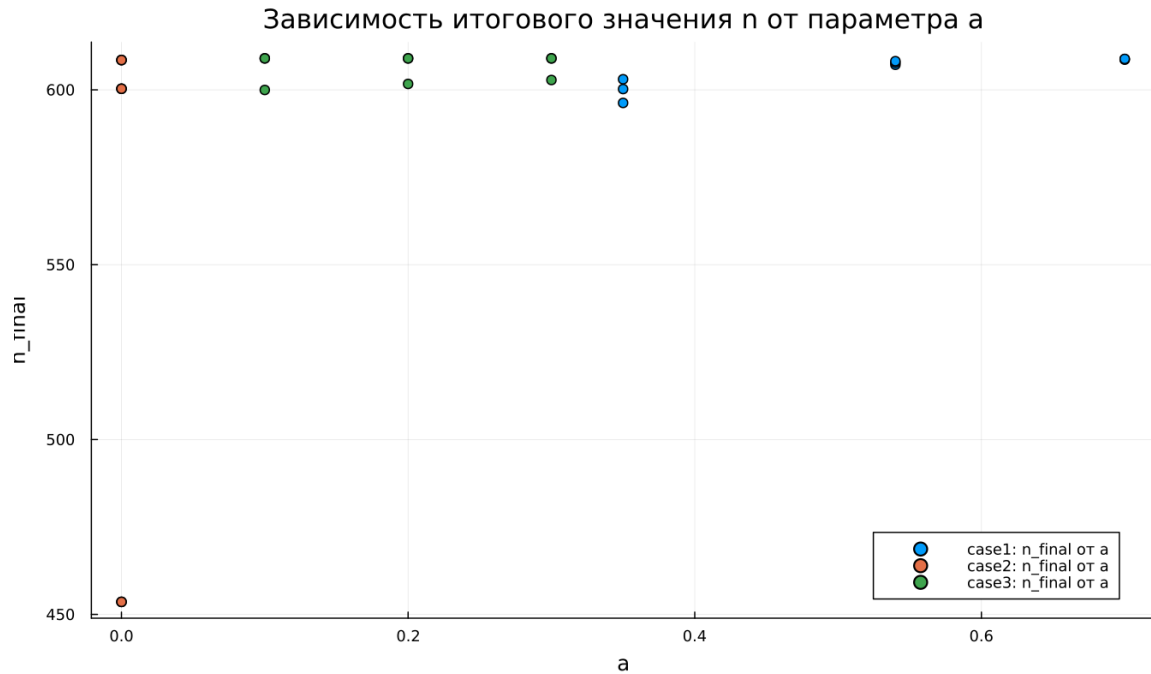


Рисунок 6.10: Зависимость итогового значения n от параметра a

На графике показана зависимость итогового значения n_{final} от параметра a для трёх моделей. Большинство точек располагается вблизи уровня $N = 609$, что указывает на достижение почти полного насыщения к концу расчётного интервала.

Для первой модели значения n_{final} близки к N при всех рассмотренных значениях параметра a . Это означает, что изменение a в выбранном диапазоне не нарушает общий характер процесса: решение продолжает стремиться к предельному уровню.

Во второй модели наблюдается один заметный выброс при малом значении a . В этом случае итоговое значение n оказывается существенно ниже N и находится примерно на уровне $n_{final} \approx 455$. Это связано с тем, что при выбранной комбинации параметров процесс не успевает выйти на насыщение за короткий временной интервал.

Для третьей модели значения n_{final} также находятся в верхней части графика и близки к N . Наличие периодических коэффициентов не приводит к заметному снижению итогового уровня при выбранных параметрах.

В целом параметр a влияет не только на итоговое значение, но и на скорость выхода к насыщению. Если интенсивность роста мала, система может не успеть достичь уровня N за заданное время моделирования.

6.33.2 Зависимость итогового значения n от параметра b

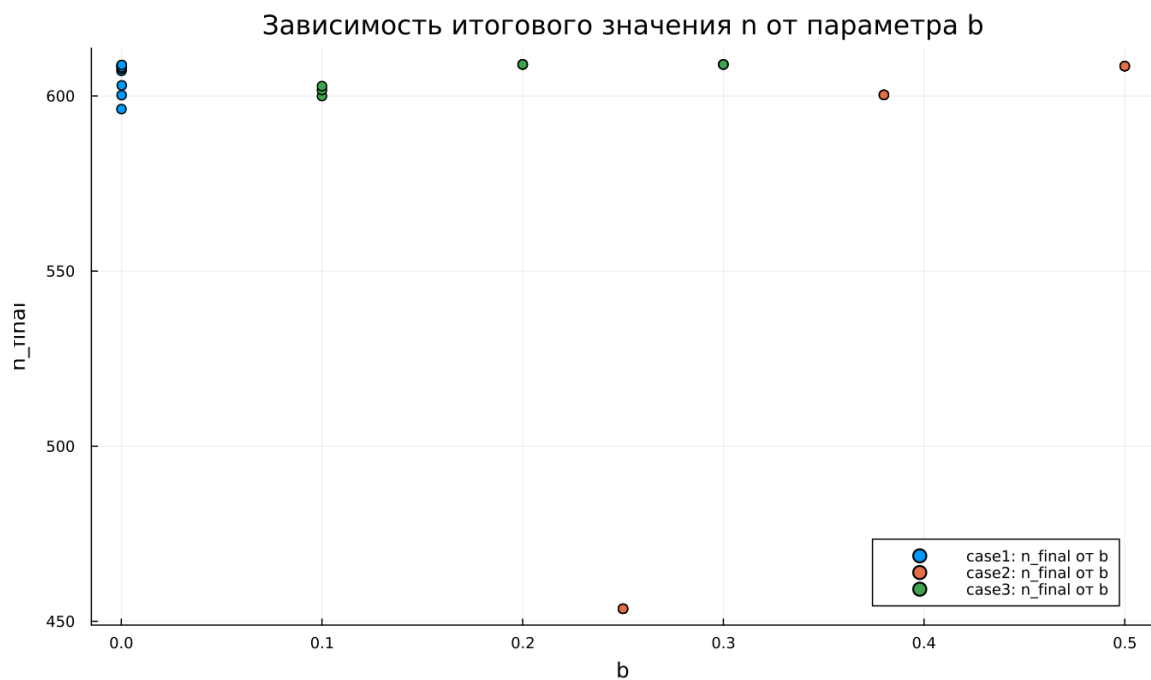


Рисунок 6.11: Зависимость итогового значения n от параметра b

График зависимости n_{final} от параметра b показывает, что для большей части экспериментов итоговое значение также располагается около $N = 609$. Это подтверждает устойчивое стремление решений к насыщению.

Для первой модели точки сгруппированы около малых значений b , поскольку в параметрическом сканировании для этой модели использовались малые значения коэффициента. Несмотря на это, итоговые значения n остаются близкими к

предельному уровню.

Во второй модели влияние параметра b выражено сильнее. При одном из значений b итоговое значение n заметно ниже уровня насыщения. Это означает, что для данной комбинации параметров рост оказывается недостаточно быстрым, и система не успевает приблизиться к N за расчётное время.

Для третьей модели точки находятся около N , что говорит о сохранении режима насыщаемого роста. Даже при наличии множителей $\cos(t)$ и $\cos(2t)$ итоговое значение остаётся близким к верхнему пределу.

График показывает, что параметр b играет ключевую роль в моделях с нелинейным усилением роста через слагаемое bn . При недостаточном значении коэффициента выход на насыщение может замедляться.

6.33.3 Зависимость максимального значения n от параметра a

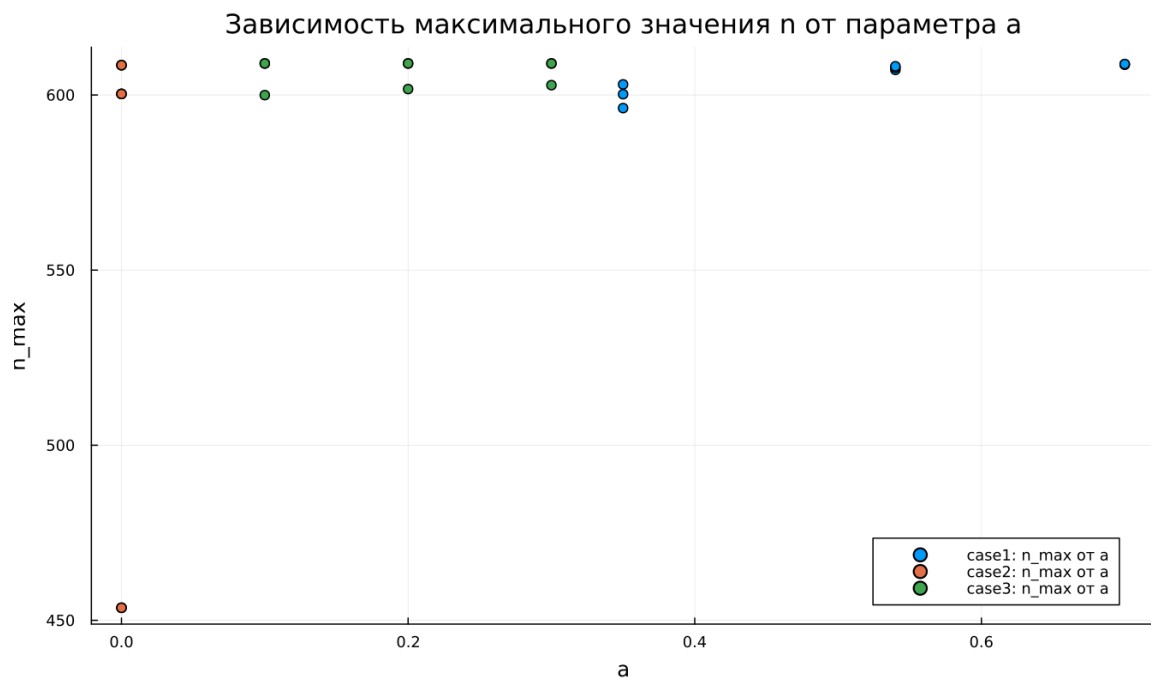


Рисунок 6.12: Зависимость максимального значения n от параметра a

На графике представлена зависимость максимального значения n_{max} от параметра a . Поскольку во всех базовых экспериментах функция $n(t)$ возрастает монотонно, величина n_{max} практически совпадает с итоговым значением n_{final} .

Для первой модели максимальные значения находятся около уровня $N = 609$. Это показывает, что решение достигает верхней границы или подходит к ней достаточно близко.

Во второй модели снова выделяется точка с меньшим значением n_{max} . Это означает, что на всём временном интервале решение не смогло приблизиться к насыщению. Причина связана не с изменением направления динамики, а с недостаточной скоростью роста на заданном интервале времени.

Для третьей модели максимальные значения сохраняются около N . Следовательно, переменные во времени коэффициенты не препятствуют достижению высокого уровня n при выбранном диапазоне параметров.

Поскольку n_{max} отражает наибольшее значение решения за время расчёта, данный график подтверждает выводы, полученные при анализе n_{final} .

6.33.4 Зависимость максимального значения n от параметра b

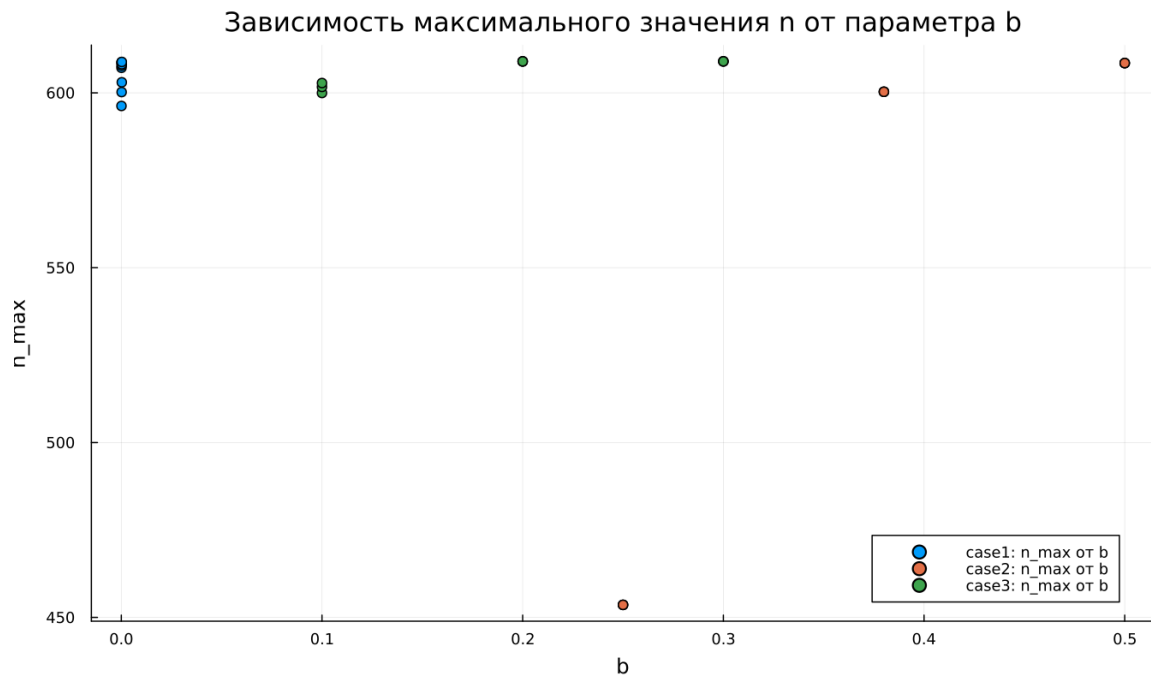


Рисунок 6.13: Зависимость максимального значения n от параметра b

График зависимости n_{max} от параметра b имеет структуру, близкую к графику $n_{final}(b)$. Это связано с монотонным ростом решений: максимум достигается в конце расчётного интервала или вблизи него.

Для первой модели значения n_{max} сосредоточены около N . Изменение параметра b в малом диапазоне не приводит к существенному снижению максимального уровня.

Во второй модели одна точка располагается значительно ниже остальных. Это показывает, что при соответствующей комбинации параметров модель не выходит на насыщение. Остальные эксперименты для второй модели дают значения около верхнего предела.

Для третьей модели все точки остаются вблизи N , что говорит о стабильном достижении насыщения. Параметр b влияет на скорость роста, но в выбранных экспериментах не меняет конечный характер динамики.

График подтверждает, что максимальный уровень n в первую очередь ограничивается величиной N , а параметры a и b определяют скорость приближения к этому пределу.

6.33.5 Зависимость финального насыщения от параметра a

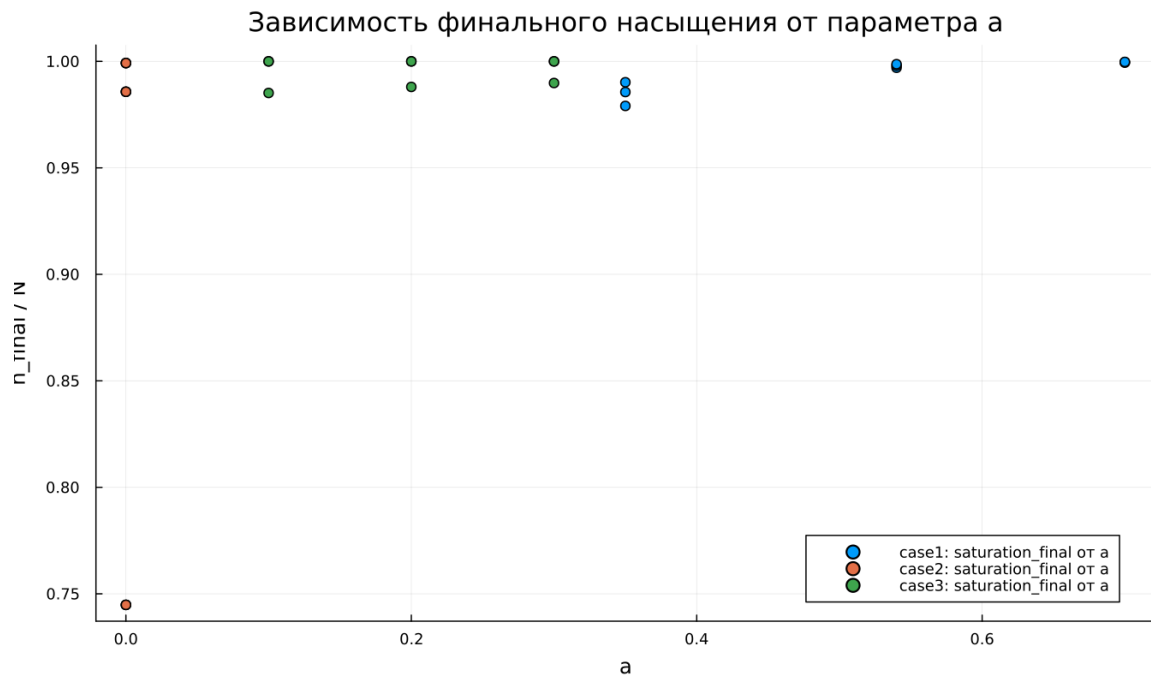


Рисунок 6.14: Зависимость финального насыщения от параметра a

На графике показана зависимость относительного итогового насыщения n_{final}/N от параметра a . Значение, близкое к 1, означает почти полное достижение предельного уровня N .

Для первой модели насыщение находится около 1 при всех рассмотренных значениях a . Это показывает, что модель стабильно достигает предельного состояния.

Во второй модели наблюдается одна точка с насыщением около 0,75. Это означает, что при данной комбинации параметров итоговое значение составляет примерно три четверти от возможного максимума. Остальные точки располагаются

значительно выше и находятся близко к полному насыщению.

Для третьей модели значения n_{final}/N находятся вблизи 1. Это указывает на достижение почти полного насыщения даже при наличии временной зависимости коэффициентов.

График удобен для сравнения моделей, поскольку нормировка на N позволяет оценивать не абсолютное значение n , а степень приближения к верхнему пределу.

6.33.6 Зависимость финального насыщения от параметра b

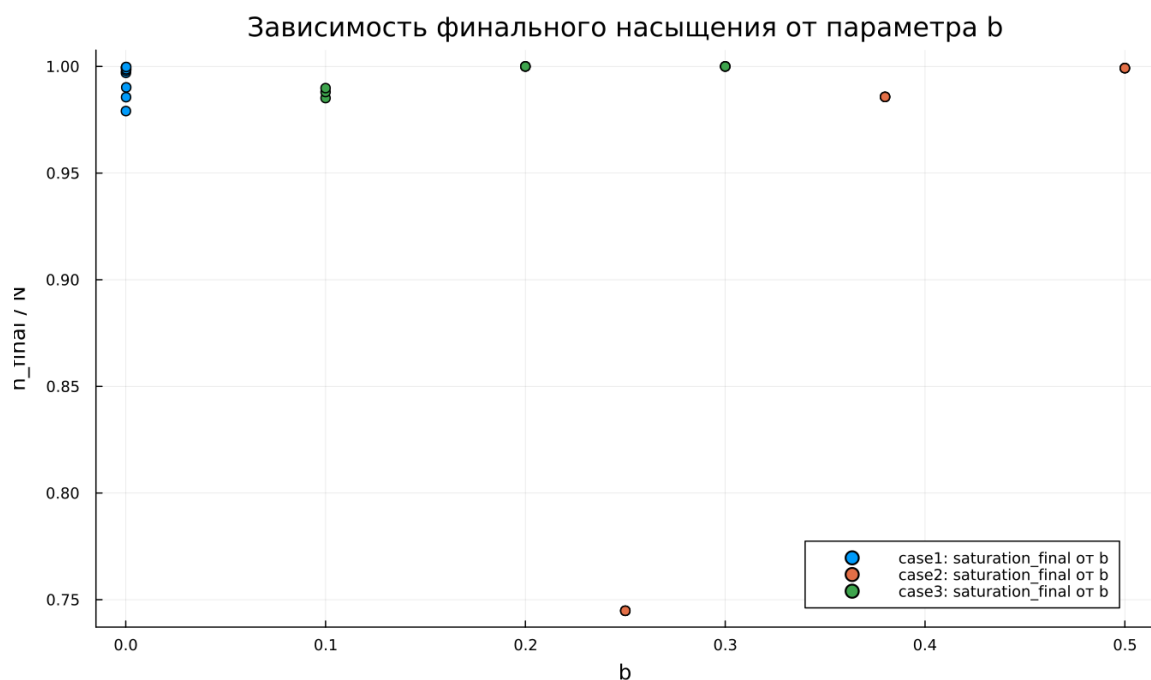


Рисунок 6.15: Зависимость финального насыщения от параметра b

График зависимости n_{final}/N от параметра b показывает, что большинство экспериментов завершается почти полным насыщением. Значения расположены вблизи 1, что соответствует достижению уровня N .

Для первой модели насыщение остаётся высоким при всех рассмотренных значениях b . Это означает, что даже малые значения параметра не мешают решению приблизиться к предельному уровню за выбранное время.

Во второй модели снова наблюдается одно пониженное значение. В этом эксперименте система достигает только около 75% от уровня N . Данный результат показывает чувствительность второй модели к сочетанию параметров и длине расчётного интервала.

Для третьей модели точки располагаются вблизи полного насыщения. Временные множители не создают заметного снижения итогового уровня.

В целом график подтверждает, что система стремится к насыщению, однако при отдельных наборах параметров время моделирования может оказаться недостаточным для выхода на уровень N .

6.34 Бенчмаркинг

6.34.1 Зависимость времени решения ODE от параметра a

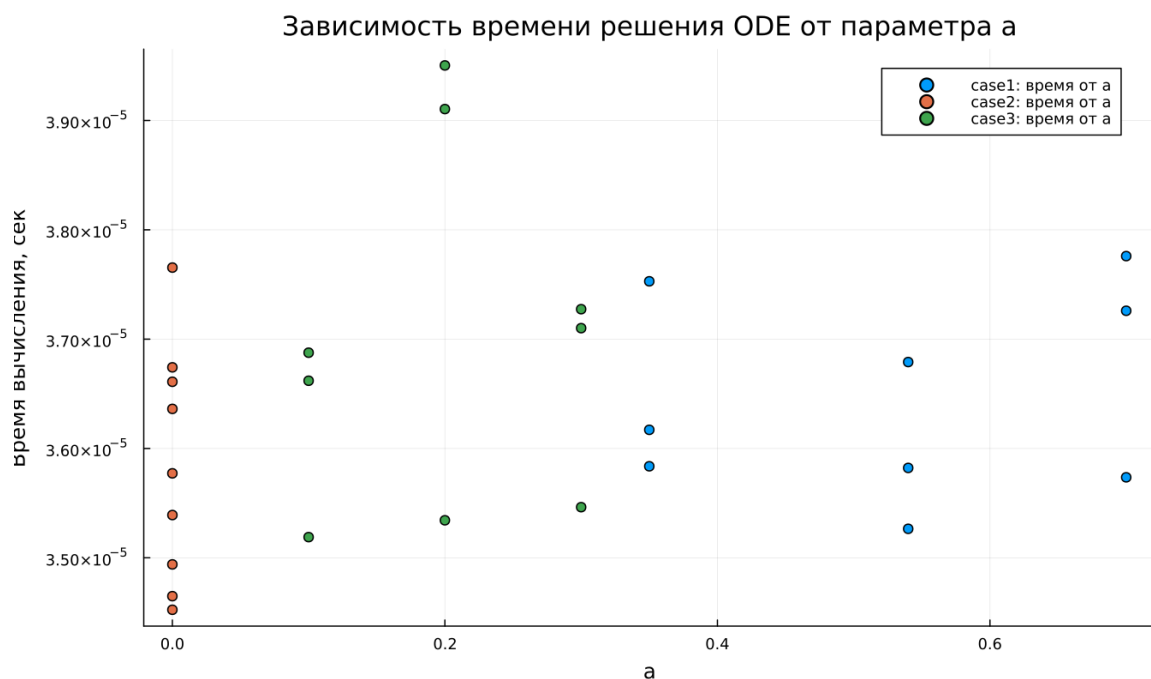


Рисунок 6.16: Зависимость времени решения ODE от параметра a

На графике показано медианное время численного решения задачи ОДУ при различных значениях параметра a . Время вычислений находится примерно в диапазоне от $3,45 \cdot 10^{-5}$ до $3,95 \cdot 10^{-5}$ секунды.

Для всех трёх моделей значения времени близки друг к другу. Это означает, что рассматриваемые задачи имеют малую вычислительную сложность, а изменение параметров не приводит к резкому росту затрат на интегрирование.

Явной монотонной зависимости времени решения от параметра a не наблюдается. Точки располагаются с небольшим разбросом, который может быть связан с особенностями работы численного метода, кэшированием, системной нагрузкой и погрешностью измерения при малых временах выполнения.

Третья модель в отдельных точках показывает немного большее время решения. Это можно объяснить более сложной правой частью уравнения, где дополнительно вычисляются функции $\cos(t)$ и $\cos(2t)$.

В целом бенчмаркинг показывает, что все три модели решаются быстро, а различия между ними по времени вычислений остаются небольшими.

6.34.2 Зависимость времени решения ODE от параметра b

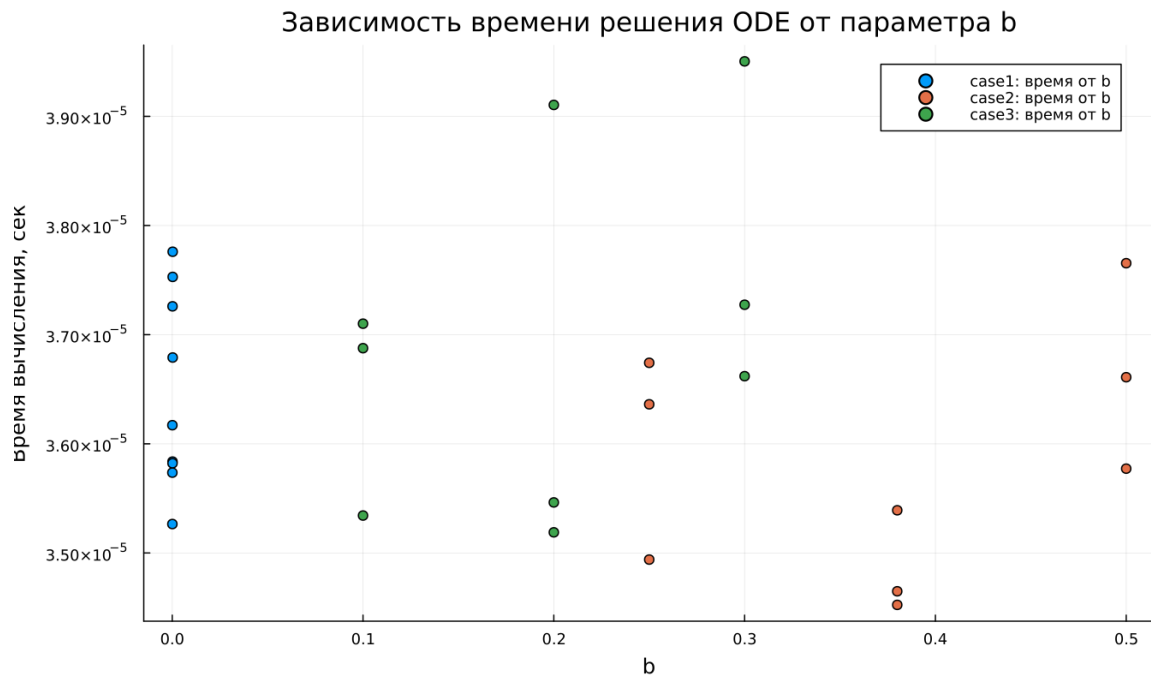


Рисунок 6.17: Зависимость времени решения ODE от параметра b

На графике представлена зависимость времени решения ОДУ от параметра b . Как и в случае с параметром a , значения времени находятся в узком диапазоне порядка 10^{-5} секунды.

Для первой модели точки сосредоточены около малых значений b , что соответствует выбранной сетке параметров. Время решения немного колеблется, но не демонстрирует устойчивого роста или убывания.

Для второй модели значения времени также изменяются с небольшим разбросом. Даже при увеличении b вычислительные затраты не возрастают существенно. Это говорит о том, что выбранный решатель справляется с задачей без заметного усложнения интегрирования.

Для третьей модели часть точек располагается выше, чем у первых двух моделей. Вероятная причина состоит в более сложной формуле правой части, содержащей тригонометрические множители. При этом различие остаётся небольшим и не

меняет общий вывод о высокой скорости расчёта.

График показывает, что параметр b влияет на динамику решения сильнее, чем на время вычислений. В рамках проведённого сканирования вычислительная стоимость остаётся стабильной для всех трёх моделей.

6.35 Выводы

1. Первая модель (case1) описывает насыщаемый рост величины $n(t)$. Решение монотонно возрастает и постепенно приближается к предельному уровню $N = 609$, не превышая его.
2. Во второй модели (case2) наблюдается наиболее резкий рост. Функция $n(t)$ имеет выраженную S-образную форму: сначала рост происходит медленно, затем ускоряется, после чего замедляется при приближении к уровню насыщения.
3. Третья модель (case3) также приводит систему к насыщению, но отличается наличием переменных во времени коэффициентов $\cos(t)$ и $\cos(2t)$. На выбранном интервале они не вызывают колебаний, поэтому решение остаётся монотонным.
4. Во всех трёх моделях предельное значение N играет роль уровня насыщения. При приближении $n(t)$ к N множитель $(N - n)$ уменьшает скорость роста, а при $n = N$ правая часть уравнения обращается в ноль.
5. Графики скорости изменения dn/dt показывают различие между моделями. В первой модели скорость максимальна в начальный момент и затем монотонно убывает. Во второй и третьей моделях скорость сначала возрастает, достигает максимума, а затем снижается почти до нуля.
6. Фазовые траектории $dn/dt(n)$ подтверждают характер динамики. Для первой модели траектория имеет убывающий вид, а для второй и третьей моделей

— форму, близкую к параболической, с внутренней точкой максимальной скорости роста.

7. Параметры a и b влияют прежде всего на скорость выхода к насыщению. При достаточно больших значениях параметров система быстро достигает уровня N , а при отдельных комбинациях параметров, особенно во второй модели, решение не успевает полностью выйти на насыщение за заданное время.
8. Метрики n_{final} и n_{max} в большинстве экспериментов близки к $N = 609$, что указывает на достижение предельного состояния. Заметные отклонения связаны не с изменением направления динамики, а с недостаточной длительностью расчётного интервала для некоторых наборов параметров.
9. Метрика $\text{saturation_final} = n_{final}/N$ позволяет оценить степень достижения насыщения. Значения, близкие к единице, показывают почти полный выход системы на предельный уровень, а пониженные значения указывают на незавершённый переходный процесс.
10. Бенчмаркинг показал, что численное решение всех трёх моделей выполняется быстро. Время вычислений находится порядка 10^{-5} секунды, а изменение параметров a и b практически не влияет на вычислительные затраты.
11. Третья модель немного сложнее с вычислительной точки зрения, поскольку правая часть содержит тригонометрические функции. Однако разница во времени решения остаётся небольшой и не оказывает существенного влияния на общую эффективность расчётов.
12. В целом все три модели описывают процесс насыщаемого роста, но различаются скоростью перехода к предельному состоянию и формой зависимости dn/dt от времени и от значения n .

Список литературы

1. Модель Мальтуса
2. Логистическая модель роста